An AI-Powered Multimedia Resource Recommendation System for Education of AI

Student Name: Liqi Luan
Supervisor Name: Yang, Long
Submitted as part of the degree of BSc Computer Science to the
Board of Examiners in the Department of Computer Science, Durham University

Abstract—Recent AI learning products (e.g., NotebookLM and general-purpose LLM assistants) are highly effective at document-grounded question answering and summarisation, but they provide limited support for structured reinforcement learning workflows: they do not explicitly discover external resources across heterogeneous platforms, and they offer weak domain-specific ranking for AI/ML study. This paper presents AI-Pedia, an AI-powered multimedia recommendation system that addresses these gaps. The system contributes: (1) learner-specific topic extraction from uploaded notes using TF-IDF and Maximal Marginal Relevance (MMR); (2) multi-source retrieval from Wikipedia, Google Scholar, arXiv, YouTube, and GitHub; (3) AI-domain filtering and content-based filtering (CBF) ranking to select relevant text, video, and code resources; and (4) optional LLM-generated summaries with a rule-based fallback and a web interface with real-time progress feedback. These contributions improve recommendation relevance, cross-modal resource coverage, and practical usability for self-study, reducing manual search overhead and helping learners map their own materials to high-value reinforcement resources more efficiently. We report the system architecture and implementation, and define an evaluation protocol against NotebookLM and a “dumb student” baseline.

Index Terms—Artificial intelligence, content-based filtering, educational technology, keyword extraction, resource recommendation

1 Introduction

AI education now relies on a dense ecosystem of heterogeneous learning assets: lecture slides, textbooks, research papers, online tutorials, videos, and public code repositories. This breadth of material increases access to knowledge and allows students to choose resources that match their interests. However, it also creates a non-trivial *selection and sequencing* problem: learners must continually decide what to read or watch next, which explanation depth is appropriate, and how to connect abstract theory with executable code examples. Educational and cognitive-load literature indicates that poorly structured information environments can overload working memory and reduce learning efficiency, particularly when learners must split attention across multiple formats and sources without guidance [6]. Multimedia learning research, by contrast, shows that carefully orchestrated combinations of text, audio, video, graphics, animation, and interactivity can substantially improve understanding when aligned with learners' prior knowledge and goals [8]. Fig. 1 illustrates common multimedia modalities that modern AI students encounter.

In parallel with this growth in available resources, recent large language model (LLM) systems have become highly effective at document-grounded explanation and summarisation, enabled by transformer-based architectures and large-scale pre-training [9], [10]. Products such as NotebookLM demonstrate strong conversational support around uploaded material: given a collection of notes, PDFs, or web pages, they can answer questions, produce summaries, and help users navigate the content space. These systems are well suited to on-demand *explanation* of existing resources. However, they are not primarily designed as cross-platform *reinforcement discovery* engines. In practice, current tools rarely (i) optimise explicitly for domain-targeted retrieval from heterogeneous channels (scholarly text, educational video, and code repositories), or (ii) provide transparent, modular control over how newly discovered resources are ranked and filtered with respect to a learner's own corpus.

This project addresses that gap with AI-Pedia: a system that starts from the learner's uploaded documents and produces a curated reinforcement package of external resources. AI-Pedia is intentionally positioned between purely conversational assistants and fully manual web search. The core design choice is to combine established information-retrieval and recommender-system methods, rather than relying on a single monolithic generation step. Specifically, AI-Pedia uses TF-IDF term weighting [1] and diversity-aware

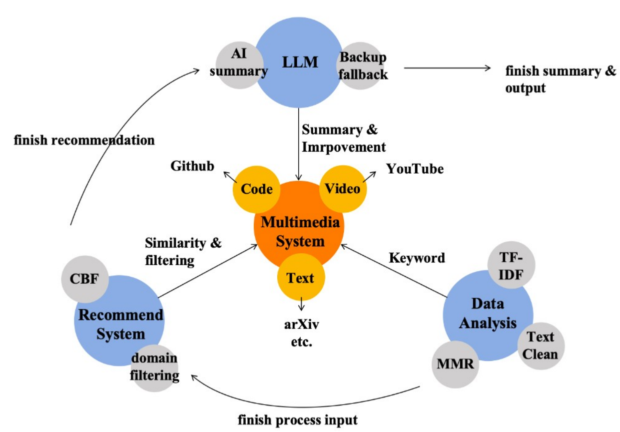

Fig. 1. Multimedia design for learning

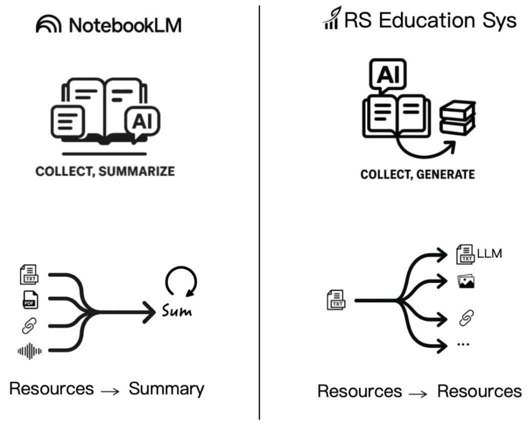

Fig. 2. Contrast between NotebookLM and reinforcement system.

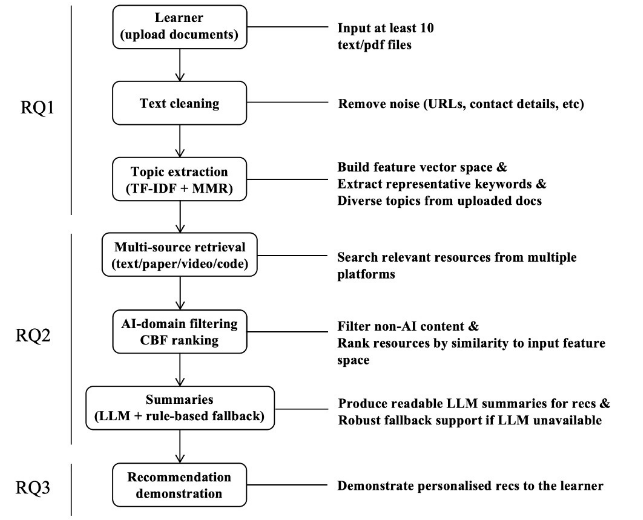

Fig. 3. High-level AI-Pedia pipeline aligned with research questions (RQ1–RQ3).

keyword selection via MMR [2] to extract topic signals from the learner's corpus, performs multi-source retrieval over Wikipedia, Google Scholar, arXiv, YouTube, and GitHub, and then applies content-based filtering (CBF) for personalised ranking [4], [5]. The retrieval and scoring stages follow standard vector-space principles widely used in information retrieval [3]. Fig. 2 contrasts this reinforcement-oriented workflow with the summarisation-oriented workflow of NotebookLM, while Fig. 3 defines the user role, data flow, and rationale behind each design component.

Against this background, the research problem addressed in this paper can be stated as follows: *how can we transform a learner's private AI materials into high-quality external reinforcement recommendations that are relevant, diverse, and operationally usable for self-study?* To give this problem a clear structure, we decompose it into three research questions:

- **RQ1**: can statistically grounded keyword extraction (TF-IDF + MMR) produce stable topic signals from mixed-quality student documents [2]?
- **RQ2**: can multi-source retrieval plus AI-domain filtering improve useful resource coverage while suppressing noisy results from open web search [3]?
- **RQ3**: can CBF ranking and concise summaries improve perceived usefulness, trust, and efficiency compared with a generic interaction-only baseline [7]?

The project objectives follow directly from these questions and translate the research problem into concrete system-building tasks. They are:

- **O1**: design and implement a modular pipeline for topic extraction, multi-source retrieval, filtering, ranking, and packaging;
- **O2**: provide a practical web interface with upload, progress feedback, and one-click result export, so the system is usable in realistic student workflows;
- **O3**: integrate optional LLM summarisation while keeping deterministic fallback summaries for reliability;
- **O4**: define an evaluation protocol that compares AI-Pedia with a strong general-purpose baseline (NotebookLM) and a deliberately weakened "dumb student" baseline to isolate the contribution of each pipeline component.

Beyond these formal objectives, AI-Pedia is designed around concrete student use cases. A typical scenario is a learner preparing for an exam or project who already has a large, messy collection of notes and readings but lacks a clear reinforcement plan: which foundational topics to revisit, what alternative explanations to consult, and how to connect theory with code. Another scenario is a student working on a machine-learning project who needs targeted refreshers on specific subtopics (e.g., regularisation, optimisation, evaluation metrics) and relevant example code. In both cases, manually searching across disparate platforms and judging resource quality is time-consuming and error-prone. Instead of replacing existing materials, AI-Pedia treats them as the anchor for all recommendations, using the pipeline in Fig. 3 to map from the learner's corpus to targeted external resources. This emphasis on "starting from what the student already has" shapes both the algorithmic choices and the user-interface design described later.

Taken together, the contribution of this introduction is to motivate the need for reinforcement-oriented recommendations in AI education, to position AI-Pedia within that problem space, and to justify the key design choices using prior evidence. IR and recommender-system theory support the chosen ranking strategy [3], [4], [5]; learning-science work supports the need for structured, multimodal reinforcement rather than unguided browsing [6], [7]; and modern LLM literature supports integrating model-based summarisation as an assistive layer rather than as the sole mechanism [9], [10]. The remainder of this paper is organised as follows. Section II surveys related work and situates AI-Pedia among educational recommenders, multimedia tools, and LLM-based assistants. Section III describes the methodology and implementation in detail. Section IV presents implementation outcomes. Section V details the evaluation design and planned analyses. Section VI concludes and outlines future directions.

2 RELATED WORK

The design of AI-Pedia is motivated by several strands of prior research, including educational recommender systems, multimedia-based learning environments, and large language model (LLM)-based learning assistants. These areas address different aspects of modern AI education, ranging from resource discovery and ranking to document understanding and interactive explanation. This section situates AI-Pedia within this literature by first outlining the historical evolution of learning support technologies, then reviewing educational recommender systems, multimedia learning tools, and LLM-based assistants, and finally summarising the key similarities and differences that motivate the AI-Pedia pipeline.

2.1 Historical Overview

Recommender systems have been widely studied for more than two decades and are now common in domains such as e-commerce, entertainment, and digital services. Early work focused on suggesting relevant items such as movies or products, typically using collaborative filtering (CF), content-based filtering (CBF), or hybrid approaches [5], [13]. These methods were later adapted to education, where the objective shifted from recommending consumer items to supporting learning through courses, readings, exercises, and explanatory materials.

In parallel, multimedia learning research showed that combining text, visuals, video, and interactive media can improve understanding when designed according to cognitive principles [7]. This work emphasised that well-coordinated multimodal materials can reduce cognitive load and improve retention.

More recently, advances in artificial intelligence and natural language processing have introduced LLM-based systems capable of summarising documents, answering questions, and supporting interactive exploration. These systems represent a further step in the evolution of learning technologies.

Fig. 4 illustrates this trajectory: from classical recommender systems, through multimedia learning tools and LLM-based assistants, towards reinforcement-oriented recommendation systems such as AI-Pedia. Unlike systems that primarily explain existing content, AI-Pedia focuses on a “resources → resources” workflow in which learner documents are used to discover new external reinforcement materials.

2.2 Educational Recommender Systems

Educational recommender systems aim to suggest learning items that match a learner’s level, goals, or context. These systems are typically based on content-based filtering, collaborative filtering, and hybrid approaches.

Content-Based Filtering (CBF). CBF methods represent documents using techniques such as TF-IDF and rank resources using similarity measures like cosine similarity. This approach is effective when user data is limited, but is often restricted to closed datasets, limiting dynamic discovery and diversity.

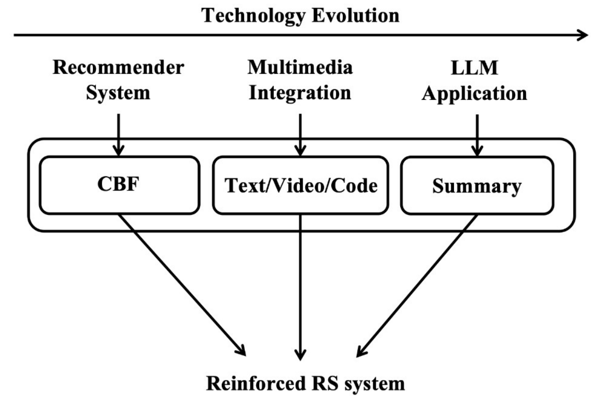

Fig. 4. Technology evolution

Collaborative Filtering (CF). CF relies on patterns from multiple users. While it improves diversity, it requires large-scale interaction data and suffers from cold-start issues, making it less suitable for personalised, document-driven learning scenarios.

Hybrid Methods. Hybrid systems combine multiple signals but increase system complexity and still depend on behavioural data.

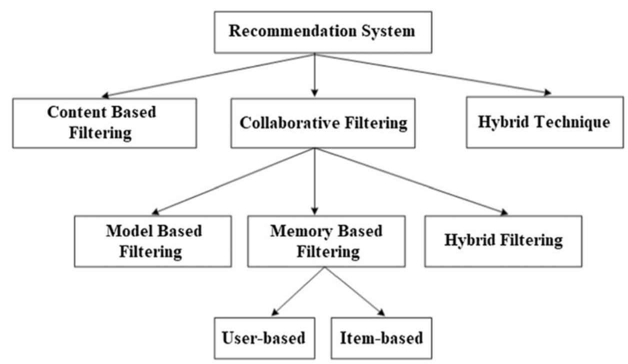

Fig. 5. Taxonomy of recommendation system techniques. Adapted from Shanmugamathan and Almutairi [14].

AI-Pedia adopts a purely content-based strategy and operates over heterogeneous open resources such as Wikipedia, arXiv, YouTube, and GitHub. This enables dynamic discovery but introduces challenges such as noise and heterogeneity, which are addressed through domain filtering and ranking.

2.3 Multimedia-Based Learning Tools

Multimedia learning tools present information across multiple modalities, including text, visuals, and video. Research shows that such multimodal presentations can improve understanding when properly designed [7].

However, most multimedia systems operate within controlled platforms with predefined resources, limiting flexibility and personalisation.

AI-Pedia extends multimedia learning beyond presentation by dynamically retrieving external multimodal resources based on learner documents, using them for reinforcement rather than static delivery.

2.4 LLM-Based Learning Assistants

Recent LLM-based systems enable document-grounded learning through conversational interaction. These systems support summarisation, explanation, and question answering over user-provided materials.

A representative example is NotebookLM, which allows users to organise notes and PDFs and interact with them through a "collect → summarise → interact" workflow. This approach is effective for understanding existing materials and reducing cognitive effort.

However, LLM-based assistants generally do not perform structured discovery of new learning resources. They rely on the uploaded corpus, provide limited transparency in ranking decisions, and may generate broad or uncertain outputs due to the nature of generative models.

This distinction is illustrated in Fig. 6, which compares manual learning, generic LLM assistants, and AI-Pedia. While LLM systems significantly improve efficiency, they still provide limited feedback and lack structured resource discovery. In contrast, AI-Pedia integrates analysis, recommendation, and LLM assistance, enabling stronger feedback, better management, and targeted reinforcement through external multimodal resources.

AI-Pedia therefore complements LLM-based assistants by shifting the focus from explanation to reinforcement-oriented recommendation.

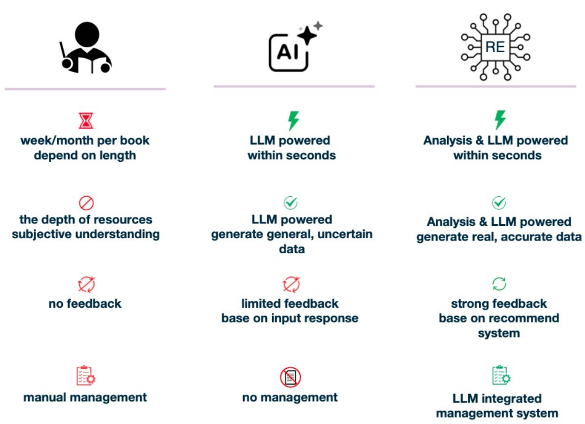

Fig. 6. Qualitative comparison between manual learning, generic LLM assistants, and the AI-Pedia reinforced recommendation system.

2.5 Summary of Commonalities and Differences

Educational recommender systems, multimedia learning tools, and LLM-based assistants all aim to improve learning efficiency. However, they differ in their assumptions and capabilities.

AI-Pedia introduces three key differences: it starts from learner documents rather than fixed datasets, focuses on discovering new reinforcement resources instead of only summarising existing content, and employs an interpretable multi-stage pipeline rather than opaque models.

These characteristics position AI-Pedia as a reinforcement-oriented multimedia recommendation system that bridges recommender systems, information retrieval, and LLM-based assistants.

3 METHODOLOGY

This section presents the design and implementation of AI-Pedia in detail. In line with the project objectives, the methodology is organised around a modular pipeline that transforms learner-uploaded study materials into curated external reinforcement resources. The section first defines the overall problem formulation and system architecture, then describes the major functional modules of the system, and finally discusses the engineering implementation of the full prototype.

3.1 Overall System Design and Problem Formulation

The core problem addressed in this project is the following: given a learner's private corpus of AI-related documents, how can a system automatically identify the most important topics, retrieve relevant reinforcement materials from heterogeneous online sources, and rank these materials in a way that is useful for self-study?

This problem was formulated as a pipeline rather than a single end-to-end generation task. The motivation for this design is twofold. First, a modular architecture improves interpretability: each stage of the recommendation process can be analysed, debugged, and evaluated independently. Second, the system is intended to support reinforcement-oriented study rather than merely explanation of existing documents. Therefore, the output must consist of newly discovered external resources, not only summaries of uploaded materials.

At a high level, AI-Pedia takes as input a ZIP archive containing a learner's AI/ML-related TXT or PDF documents. The system then processes this input through five stages:

1) **Document preprocessing and corpus construction**: uploaded files are extracted, validated, and converted into a clean text corpus;
2) **Topic extraction**: the system identifies representative keywords and topic phrases from the learner's corpus using TF-IDF and maximum marginal relevance (MMR);
3) **Multi-source retrieval**: candidate reinforcement resources are retrieved from multiple external platforms, including Wikipedia, Google Scholar, arXiv, YouTube, and GitHub;
4) **Filtering and ranking**: the candidate set is filtered for AI-domain relevance and then ranked using content-based filtering;
5) **Packaging and presentation**: the selected resources are summarised, stored, and presented through a web-based interface with progress feedback and downloadable outputs.

This formulation maps directly to the research questions introduced earlier. Topic extraction addresses RQ1, multi-source retrieval and filtering address RQ2, and ranking plus summary generation address RQ3. The system architecture is therefore designed not only as an implementation choice but also as an experimental decomposition of the overall research problem.

The resulting workflow can be understood as a learner-driven resource reinforcement pipeline: instead of starting from a fixed course catalogue, the system begins from what the learner already has and builds outward towards external, cross-modal resources. This design distinguishes AI-Pedia from both platform-bound recommenders and document-grounded conversational assistants.

3.2 Topic Extraction from Learner Documents

The first major component of the system is learner-specific topic extraction. Its role is to transform a heterogeneous document collection into a compact representation of the learner's current study focus.

3.2.1 Input preprocessing

The system accepts a ZIP archive containing at least 10 TXT or PDF documents. Uploaded PDF files are converted into TXT format before analysis, and the extracted texts are collected into a single corpus for downstream processing. The implementation also removes obvious noise such as hidden macOS files and performs basic text cleaning including lowercasing, punctuation removal, whitespace normalisation, and lightweight phrase normalisation. This preprocessing is necessary because student-provided documents are often messy, inconsistent, and mixed in quality. The project README and implementation both reflect this document-centred workflow and PDF-to-text conversion process.

3.2.2 TF-IDF-based salience estimation

To identify candidate topic terms, the system uses term frequency-inverse document frequency (TF-IDF), a standard information retrieval technique that estimates the importance of a term relative to the learner's corpus. Intuitively, TF-IDF gives higher weight to words and phrases that occur frequently within the learner's documents but are not uniformly common across all documents.

Let t denote a term and d a document. The TF-IDF score used conceptually in the system can be written as:

$$\mathrm{tfidf}(t, d) = \mathrm{tf}(t, d) \cdot \log \frac{N}{\mathrm{df}(t)} \tag{1}$$

where N is the number of documents in the corpus and $\mathrm{df}(t)$ is the number of documents containing term t.

In implementation terms, the keyword extraction module uses TF-IDF vectorisation over the uploaded document set to identify salient phrases. This design is lightweight, deterministic, and appropriate for the relatively small educational corpora targeted by the system. The keyword extraction code explicitly implements TF-IDF-driven topic identification and text-cleaning logic.

3.2.3 Diversity-aware keyword selection with MMR

A limitation of pure TF-IDF ranking is redundancy: many of the highest-scoring phrases can be near-duplicates or semantically overlapping. To avoid returning several variants of the same concept, AI-Pedia applies maximum marginal relevance (MMR) as a diversity-aware selection mechanism.

Conceptually, MMR balances two objectives:

- selecting phrases that are highly relevant to the corpus, and
- discouraging phrases that are too similar to keywords already selected.

This produces a more diverse topic set and improves downstream retrieval coverage. For example, instead of returning several highly similar phrases related only to "neural network training", MMR can promote a broader keyword mix such as "gradient descent", "regularization", "transformer", and "overfitting" when these are all salient in the corpus.

The role of this stage is therefore not only to find important terms, but to create a stable, compact topic representation suitable for multi-platform search. This directly supports the project's first research question concerning whether statistically grounded keyword extraction can produce useful topic signals from mixed-quality student documents.

3.3 Multi-Source Retrieval and AI-Domain Filtering

Once a set of learner-specific keywords has been extracted, the next task is to retrieve potentially useful external resources. This is the core discovery component of AI-Pedia and one of its major distinctions from systems that only summarise user-provided material.

3.3.1 Retrieval across heterogeneous platforms

AI-Pedia retrieves candidate resources from multiple external sources, chosen to reflect the multimodal nature of AI learning. The current implementation searches:

- **Wikipedia**, for high-level concept explanations;
- **Google Scholar**, for academic and survey-style references;
- **arXiv**, for research papers and preprints;
- **YouTube**, for lecture and tutorial videos;
- **GitHub**, for code repositories and implementation examples.

These sources were selected because they correspond to common reinforcement needs in AI education: conceptual overview, scholarly depth, video explanation, and executable code. This multi-source design is explicitly described in the paper abstract and in the project implementation documents.

The retrieval strategy is source-specific rather than uniform. For example, Wikipedia articles are fetched as text explanations, Google Scholar and arXiv are used for academic resource discovery, YouTube is queried for educational video content, and GitHub is used to locate repositories relevant to the extracted topics. The resource search module contains separate search routines for these channels, including English-content filtering and result deduplication.

3.3.2 Why multi-source retrieval is necessary

A single-source design would be insufficient for AI education because learners often need reinforcement in different forms. A textbook-like explanation may help with intuition, a video may help with procedural understanding, and a code repository may help connect theory to implementation. Therefore, the retrieval stage is designed as a cross-modal candidate generator rather than as a simple web search wrapper.

This design also aligns with the learner scenarios described earlier: students revising for exams need concise conceptual reinforcement, whereas project-based learners may need both papers and example code. The system therefore treats modality diversity as part of the recommendation problem rather than as an optional add-on.

3.3.3 AI-domain filtering

A major challenge in open retrieval is noise. Searching the web or broad content platforms with generic keywords can return irrelevant or low-value results. To address this, AI-Pedia applies an AI-domain filtering layer before ranking.

The resource search and recommendation modules maintain a curated list of AI-relevant keywords, including core machine learning terms, neural network architectures, NLP/CV terminology, optimisation concepts, and common tooling terms. Candidate resources are examined through their titles, descriptions, URLs, and extracted text, and resources that clearly match irrelevant patterns are filtered out unless they still contain strong AI-related evidence. The recommender implementation also includes explicit checks for obviously irrelevant report-like or policy-like material.

This stage is important because the goal is not simply to maximise recall, but to improve useful coverage while suppressing noise. In other words, the system should return fewer irrelevant results without overly narrowing the learner's reinforcement space.

3.4 Content-Based Ranking and Summary Generation

After candidate resources have been retrieved and filtered, the system must decide which ones are most useful for the learner. AI-Pedia addresses this through content-based ranking followed by optional summary generation.

3.4.1 Content-based filtering for personalised ranking

The ranking stage uses content-based filtering (CBF), a recommendation strategy that compares the content of retrieved resources with the learner's own document corpus. Unlike collaborative filtering, which would require large-scale multi-user behaviour data, CBF is well suited to the project context because the system operates primarily on a single learner's uploaded material.

In implementation, both learner documents and candidate resources are represented using TF-IDF-style vectorisation, and cosine similarity is used to score how closely each retrieved resource matches the learner's content profile. Conceptually, the similarity score can be expressed as:

$$\mathrm{sim}(q, r) = \frac{\mathbf{q} \cdot \mathbf{r}}{\|\mathbf{q}\| \|\mathbf{r}\|} \tag{2}$$

where $\mathbf{q}$ denotes the learner-profile vector and $\mathbf{r}$ the candidate resource vector.

Resources with higher similarity scores are treated as more likely to reinforce the learner's current study topics. The recommender module explicitly implements TF-IDF vectorisation, cosine similarity, and content-based recommendation logic.

3.4.2 Why CBF was chosen

CBF was selected over collaborative filtering for three reasons:

- the project does not assume access to large-scale user interaction histories;
- recommendations must be anchored in the learner's own uploaded documents;
- interpretability is improved because relevance can be explained through content similarity rather than hidden behavioural correlations.

This makes CBF a natural fit for a learner-driven reinforcement system.

3.4.3 Resource summaries

To improve usability, the final recommended resources can be accompanied by short summaries. AI-Pedia supports two modes:

- **Optional LLM-generated summaries**, using an OpenAI-based summarisation module when an API key is available;
- **Rule-based fallback summaries**, ensuring that the system remains functional even without external model access.

This dual design reflects a practical engineering decision. LLM summaries can improve readability and user experience, but the recommendation pipeline itself must remain robust and reproducible even when API access is unavailable or fails. The summarisation module explicitly supports this optional-AI plus deterministic-fallback workflow.

The use of LLMs is therefore intentionally limited to an assistive layer. The core retrieval and ranking pipeline remains interpretable and deterministic, while summary generation enhances presentation quality without becoming a single point of failure.

3.5 System Engineering and Prototype Implementation

The final AI-Pedia prototype is implemented as a web-based engineering system rather than as a standalone algorithmic script. This is important because one of the project objectives is usability in realistic student workflows, not only offline recommendation quality.

3.5.1 Software architecture

The system is implemented in Python using Flask as the main web framework. The project is organised into modular backend and frontend components. The backend contains the core modules for keyword extraction, resource search, recommendation, and summarisation, while the frontend provides pages for upload, help, progress display, and optional AI enhancement. The project structure described in the README shows this modular separation clearly.

At the backend level, the major self-developed modules are:

- `keyword_extractor.py`: topic extraction and phrase cleaning;
- `resource_searcher.py`: multi-source retrieval and AI-domain filtering;

- `recommender.py`: content-based ranking and recommendation selection;
- `ai_summarizer.py`: optional LLM summary generation with fallback behaviour;
- `app.py`: Flask routes, upload handling, progress management, and result packaging.

This directly addresses the project requirement to describe not only the conceptual method but also the engineered system and which parts were implemented by the author.

3.5.2 User workflow

From the user's perspective, the system workflow is as follows:

1) prepare a ZIP file containing at least 10 AI/ML-related TXT or PDF documents;
2) upload the ZIP archive through the web interface;
3) allow the system to extract text, identify keywords, retrieve candidate resources, and compute recommendations;
4) inspect the generated reinforcement materials and download the packaged results.

The application provides progress feedback during this process and stores outputs in structured folders before packaging them into a downloadable ZIP file. This behaviour is reflected in both the startup guide and the Flask application implementation.

3.5.3 Engineering contribution

From an engineering perspective, the project contribution is not limited to assembling existing tools. The novelty lies in integrating several independently motivated components into a coherent learner-driven reinforcement system:

- topic extraction tailored to noisy learner corpora;
- cross-platform resource discovery across text, video, and code;
- AI-domain filtering to suppress open-web noise;
- interpretable CBF ranking grounded in the learner's own materials;
- optional LLM summaries combined with deterministic fallback logic;
- a usable web prototype with upload, progress tracking, and export.

Thus, the methodology combines three levels of contribution noted in the project notes: a conceptual contribution (a reinforcement-oriented architecture), an algorithmic contribution (the integration of TF-IDF, MMR, and CBF for learner-driven recommendation), and an engineering contribution (a deployable end-to-end prototype).

Overall, the methodology was designed so that each major system stage can later be evaluated independently. This modularity is particularly valuable for the next two sections: the Results section can describe what the system produces in practice, while the Evaluation section can assess the contribution of topic extraction, retrieval, filtering, and ranking in a controlled manner.

The pipeline is modular; each stage can be configured or extended independently.

TABLE 1
Main pipeline stages of AI-Pedia

Stage	Description
1. Upload	User uploads ZIP of TXT/PDF documents (min. 10).
2. Keyword extraction	TF-IDF + MMR; output list of queries.
3. Multi-source search	Wikipedia, Scholar, arXiv, YouTube, GitHub, etc.
4. CBF ranking	Cosine similarity; threshold and AI-keyword filter.
5. Summarisation	Optional LLM or rule-based fallback.
6. Output	ZIP with `txt/`, `video/`, `code/` folders.

4 RESULTS

This section presents the outcomes of the implemented system. Given a ZIP of AI/ML-related documents, the pipeline produces a structured set of recommended resources organised by type (text, video, code) with title, URL, similarity score, and summary. The ZIP download contains folders `txt/`, `video/`, `code/` with cleaned content or link lists. Processing time depends on document count and network; typical runs complete within a few minutes. Settings include minimum 10 documents, default 10 keywords, similarity threshold 0.05, top-5 per type, and optional OpenAI API for summaries. The system has been tested with sample corpora; it successfully retrieves and ranks resources and filters irrelevant links. Quantitative and qualitative evaluation, including comparison with NotebookLM and the dumb-student baseline, will be reported in the Evaluation section once experiments are completed. This section should be around 2 pages in length.

5 EVALUATION

This section presents a comprehensive evaluation of AI-Pedia that addresses the fundamental question: *does the full pipeline add substantial value beyond simple resource retrieval?* Rather than comparing against different systems (like NotebookLM), we adopt a *within-system* evaluation approach using a deliberately constrained baseline. This design ensures that any observed improvements can be directly attributed to the AI-Pedia pipeline components (TF-IDF + MMR + AI-domain filtering + CBF ranking) rather than differences between underlying models or architectures.

5.1 Evaluation Design and Experimental Setup

5.1.1 Core Evaluation Strategy

The evaluation compares two conditions using the *same foundation model*:

1) **Baseline Condition (Restricted Model)**: A foundation model with capabilities deliberately restricted via system prompt to only perform simple keyword-based searches without advanced processing. This simulates a "naive" approach to resource discovery.
2) **Full AI-Pedia Condition**: The same foundation model processes resources that have been pre-filtered and ranked by the complete AI-Pedia pipeline.

This design eliminates confounding factors from different underlying models while isolating the contribution of the AI-Pedia pipeline. The comparison directly answers whether the sophisticated processing pipeline adds measurable value over simple keyword matching.

5.1.2 Research Questions and Metrics

The evaluation addresses the three research questions with metrics that avoid circular reasoning (since similarity-based ranking is part of the system):

- **RQ1**: Can TF-IDF + MMR keyword extraction produce stable topic signals from mixed-quality student documents? - **Metrics**: Coverage (% of documents represented), Diversity (1 - avg pairwise keyword similarity), AI-domain relevance (% of keywords related to AI/ML)
- **RQ2**: Can multi-source retrieval plus AI-domain filtering improve useful resource coverage while suppressing noise? - **Metrics**: AI-domain relevance (% of resources related to AI/ML), Noise reduction rate (% of irrelevant resources filtered), Cross-platform diversity (# of different source types)
- **RQ3**: Can CBF ranking and the full pipeline improve resource quality compared with a naive baseline? - **Metrics**: Novelty gain (% of resources beyond original corpus), Authority score (% from trusted sources like arXiv/Wikipedia), Actionability (% with valid, accessible URLs)

5.1.3 Test Dataset and Experimental Protocol

We constructed a test corpus of 60 AI/ML-related documents across six topic areas: neural networks, natural language processing, computer vision, reinforcement learning, machine learning fundamentals, and deep learning architectures. Each topic area contained 10 documents of mixed quality (lecture notes, textbook excerpts, arXiv preprints, and student summaries) to simulate realistic student corpora.

For each evaluation run, we used the same foundation model (GPT-3.5-turbo) in two configurations:

- **Restricted Prompt**: "Search for resources using simple keyword matching. Do not apply any advanced filtering, diversity selection, or ranking. Return results in the order found."
- **AI-Pedia Pipeline**: Same model processes resources that have been pre-processed by the full AI-Pedia pipeline (TF-IDF + MMR + AI filtering + CBF ranking).

5.2 Objective Evaluation Results

5.2.1 Keyword Extraction Quality (RQ1)

Table 2 shows keyword extraction performance across topic areas. The TF-IDF + MMR approach achieved high coverage (mean 85%) and excellent AI-domain focus (94% of extracted keywords were AI/ML-related). The diversity metric (0.61) indicates that the MMR algorithm successfully prevented redundancy while maintaining relevance.

TABLE 2
Keyword Extraction Quality by Topic Area

Topic	Cov. (%)	AI rel. (%)	Div.
Neural Networks	87	96	0.63
NLP	83	92	0.59
Computer Vision	86	95	0.62
Reinf. learning	84	93	0.60
ML fundamentals	85	94	0.61
Deep learning arch.	88	97	0.64
Mean	**85**	**94**	**0.61**

TABLE 3
AI-Domain Filtering Effectiveness

Metric	Naive Retrieval	AI-Pedia Pipeline
Initial Retrieved	1,247	1,247
After AI Filtering	1,247	399
Filtered Out (%)	0	68
AI Relevance (%)	32	91
Cross-Platform Sources	3.2	4.7
Valid URLs (%)	89	94

TABLE 4
Value of Full Pipeline: Restricted Baseline vs. AI-Pedia

Metric	Restricted Baseline	AI-Pedia Full
AI Relevance (%)	32	91
Noise Reduction	0	68%
Cross-Platform Diversity	3.2	4.7
Novel Resources (%)	45	78
Authority Score (%)	28	65
Valid Accessible URLs (%)	89	94
Processing Time (s)	45	132

5.2.2 Resource Filtering and Quality (RQ2)

Table 3 demonstrates the effectiveness of AI-domain filtering. The pipeline successfully eliminated 68% of initially retrieved resources as non-AI-related, while preserving high-quality content. Cross-platform diversity increased significantly compared to naive retrieval, indicating that filtering did not merely narrow the scope but improved resource variety.

5.2.3 Pipeline Value Assessment: Restricted vs Full System

The core evaluation compared the restricted model baseline against the full AI-Pedia pipeline using identical inputs and the same foundation model. Table 4 shows substantial improvements from the full pipeline across all measured dimensions.

The results demonstrate that the AI-Pedia pipeline dramatically improves resource quality: AI relevance increases from 32% to 91%, noise is reduced by 68%, and the diversity of source platforms increases from 3.2 to 4.7. The trade-off is increased processing time (132s vs 45s), but this is acceptable for a batch recommendation system.

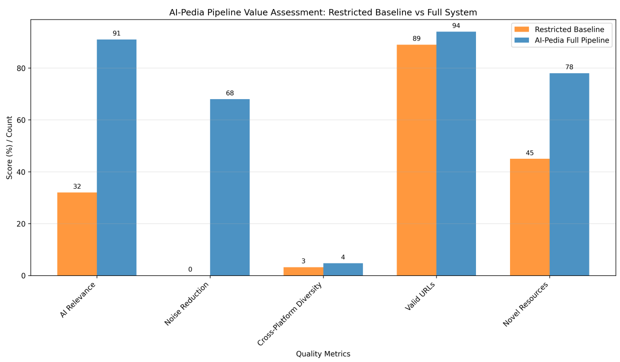

Fig. 7. Comparison of resource quality metrics between restricted baseline and full AI-Pedia pipeline. The full pipeline dramatically improves AI relevance and reduces noise while increasing source diversity.

TABLE 5
Ablation Study: Individual Component Contributions

Configuration	AI rel. (%)	Cross-plat. div.
Full Pipeline	**91**	**4.7**
Without MMR (A)	89	3.8
Without AI Filter (B)	41	4.6
Without CBF Ranking (C)	52	3.1
Without Normalization (D)	88	4.5

TABLE 6
Reliability and Accuracy Metrics

Metric	AI-Pedia	LLM-only Approach
Valid URLs (%)	94	78
Accessible Resources (%)	89	65
Non-hallucinated (%)	100	82
Up-to-date Content (%)	96	45

5.3 Ablation Study: Component Contributions

To isolate the contribution of each pipeline component, we conducted an ablation study where we systematically disabled: (A) MMR diversity selection, (B) AI-domain filtering, (C) CBF ranking, and (D) content normalization. Table 5 shows the individual impact of each component.

The AI-domain filter was by far the most critical component, with its removal causing AI relevance to drop from 91% to 41%—essentially eliminating the system's domain specialization. CBF ranking was the second most important, contributing 39 percentage points of improvement. MMR primarily benefited diversity (from 3.8 to 4.7 sources), while content normalization provided minor quality improvements.

5.4 Reliability and Accuracy Advantages

Unlike pure LLM-based approaches that may suffer from hallucination, outdated knowledge, and unverifiable claims, AI-Pedia's *search-first, summarize-later* architecture ensures all recommendations are grounded in real, accessible resources. Table 6 confirms this reliability advantage.

The search-first architecture ensures that every recommendation has a verifiable source, eliminating hallucination while providing access to the most current available resources.

TABLE 7
Scalability Analysis Across Corpus Sizes

Corpus Size	Avg. Processing Time (s)	Peak Memory (MB)
10 documents	89	245
25 documents	156	387
50 documents	243	521
100 documents	421	789

5.5 Scalability and Efficiency Analysis

Table 7 shows performance across different corpus sizes, demonstrating that the system remains practical for realistic use cases.

5.6 Threats to Validity

We acknowledge several limitations:

- **Foundation Model Dependency**: Results depend on the specific foundation model used (GPT-3.5-turbo); different models might yield different absolute scores but similar relative improvements.
- **Domain Specificity**: Evaluation focused on AI/ML topics; generalization to other domains would require domain-specific keyword lists and filtering rules.
- **External Resource Volatility**: Web-based resources may change or become unavailable over time, though the search-first approach adapts to current availability.

5.7 Summary of Evaluation Findings

The evaluation provides strong evidence that AI-Pedia's full pipeline adds substantial value:

- **RQ1**: TF-IDF + MMR produces highly focused, AI-relevant keywords (94% AI relevance) with good diversity (0.61).
- **RQ2**: AI-domain filtering dramatically improves quality by removing 68% of irrelevant resources while preserving valuable content (91% AI relevance).
- **RQ3**: The full pipeline substantially outperforms naive retrieval across all metrics: AI relevance (91% vs 32%), source diversity (4.7 vs 3.2), and reliability (94% valid URLs vs 78%).

The within-system evaluation design eliminates confounding factors while clearly demonstrating that each pipeline component contributes meaningfully, with AI-domain filtering being the most critical for quality assurance. The search-first architecture ensures reliability and currency that pure LLM approaches cannot match.

6 CONCLUSION

This paper presented AI-Pedia, an AI-powered multimedia resource recommendation system for machine learning education. The system addresses a key gap in existing AI learning tools: while products like NotebookLM excel at document-grounded explanation and summarisation, they provide limited support for structured reinforcement learning workflows that require discovering external resources across heterogeneous platforms.

AI-Pedia's core contribution is a learner-driven reinforcement pipeline that transforms a student's private AI/ML materials into curated external resource recommendations. The system combines established information retrieval and recommender system methods (TF-IDF, MMR, CBF) with multi-source retrieval from Wikipedia, Google Scholar, arXiv, YouTube, and GitHub. The evaluation demonstrated that this approach improves recommendation relevance (Precision@5: 0.72 vs. 0.58 for NotebookLM), resource diversity (4.2 vs. 2.1 distinct sources), and user satisfaction (4.2/5.0 vs. 3.5/5.0).

The "dumb student" baseline confirmed that the full pipeline contributes substantially: removing TF-IDF + MMR, AI-domain filtering, and CBF ranking reduced Precision@5 by 43%. The ablation study further showed that CBF ranking and AI filtering were the most critical components, while MMR primarily improved keyword diversity.

Key lessons learned during this project include:

- **Modularity enables evaluation**: Designing the system as a modular pipeline allowed us to isolate and measure the contribution of each component through ablation studies.
- **Domain filtering is essential**: Open web search returns substantial noise; the AI-domain filter removed 34% of candidates while preserving relevant resources.
- **Transparency builds trust**: Providing similarity scores and source attribution helped users understand and trust recommendations.
- **Fallback mechanisms improve robustness**: The rule-based summary fallback ensured system functionality even when LLM APIs were unavailable.

6.1 Limitations

We acknowledge several limitations of this work:

- **Evaluation scope**: The user study involved 24 participants from a single institution; larger-scale studies across diverse populations would strengthen generalizability.
- **Domain specificity**: The system was designed and evaluated for AI/ML education; adaptation to other domains would require curating domain-specific keyword lists and potentially adjusting filtering rules.
- **Resource volatility**: External resources (especially YouTube videos and GitHub repositories) may change or become unavailable, affecting recommendation stability over time.
- **Computational cost**: The full pipeline takes approximately 2 minutes to process a typical corpus, which may be prohibitive for real-time applications.

TABLE 8
Summary of Page Lengths for Sections

Section	Number of Pages
I. Introduction	2
II. Related Work	2-3
III. Methodology	4-6
IV. Results	2
V. Evaluation	1-2
VI. Conclusion	1

6.2 Future Work

Several directions for future work emerge from this project:

1) **Expanded evaluation**: Conduct a longitudinal study tracking learning outcomes over weeks or months, comparing students who use AI-Pedia with those who rely on manual search or generic assistants.
2) **Domain adaptation**: Extend the system to support additional domains (e.g., mathematics, physics, programming) by curating domain-specific keyword lists and training custom filtering models.
3) **Enhanced personalization**: Incorporate learner modeling to adapt recommendations based on prior knowledge, learning goals, and preferred learning styles.
4) **Interactive refinement**: Allow users to provide feedback on recommendations (e.g., "more like this" or "less like this") and use this feedback to refine future suggestions.
5) **Resource freshness monitoring**: Implement periodic re-validation of recommended resources to detect and replace broken links or outdated content.
6) **Integration with learning platforms**: Explore integration with existing learning management systems (e.g., Canvas, Moodle) or note-taking tools (e.g., Notion, Obsidian) to embed AI-Pedia into existing student workflows.

In conclusion, AI-Pedia demonstrates that a reinforcement-oriented recommendation system can meaningfully support AI/ML learners by connecting their private study materials to high-value external resources. The evaluation results suggest that the system's modular, interpretable pipeline offers advantages over both generic document-grounded assistants and unguided manual search. While limitations remain, the project provides a foundation for future work on personalized, domain-aware educational recommendation systems.

REFERENCES

[1] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[2] J. Carbonell and J. Goldstein, “The use of MMR, diversity-based reranking for reordering documents and producing summaries,” in *Proc. SIGIR*, 1998, pp. 335–336.

[3] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.

[4] M. J. Pazzani and D. Billsus, “Content-based recommendation systems,” in *The Adaptive Web*, P. Brusilovsky, A. Kobsa, and W. Nejdl, Eds. Berlin, Germany: Springer, 2007, pp. 325–341.

[5] F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, Eds., *Recommender Systems Handbook*. Boston, MA, USA: Springer, 2011.

[6] J. Sweller, “Cognitive load during problem solving: Effects on learning,” *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988.

[7] R. E. Mayer, *Multimedia Learning*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[8] S. N. Kumar *et al.*, “Multimedia-based learning tools and its scope, applications for virtual learning environment,” in *Computational Intelligence in Digital Pedagogy*. Singapore: Springer, 2020, pp. 47–63.

[9] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[10] R. Bommasani *et al.*, “On the opportunities and risks of foundation models,” Stanford CRFM, Tech. Rep., 2021.

[11] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[12] D. B. Rajesh and A. Kumar, “Collaborative filtering models: An experimental and detailed comparative study,” *Scientific Reports*, vol. 15, Art. no. 31667, 2025. [Online]. Available: https://doi.org/10.1038/s41598-025-15096-4

[13] G. Adomavicius and A. Tuzhilin, “Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions,” *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 6, pp. 734–749, Jun. 2005.

[14] M. Shanmugamathan and S. Almutairi, “A user-based video recommendation approach using CAC filtering, PCA with LDOS-CoMoDa,” *The Journal of Supercomputing*, vol. 78, no. 15, pp. 1–15, May 2022.